Received 11 April 2026; revised 16 May 2026; accepted 19 May 2026. Date of publication 26 May 2026; date of current version 8 June 2026.

Digital Object Identifier 10.1109/OJCOMS.2026.3696826

On the Design and Implementation and Evaluation of Euclidean Geometry Quasi-Cyclic LDPC Codes for NAND Flash Memories

G. ACHALA[1] (Graduate Student Member, IEEE), U. SHRIPATHI ACHARYA[1] (Senior Member, IEEE), PATHIPATI SRIHARI[1] (Senior Member, IEEE), AND LINGA REDDY CENKERAMADDI[2] (Senior Member, IEEE)

[1]National Institute of Technology, Karnataka (NIT-K), Surathkal 575025, India
[2]Department of Information and Communication Technology, University of Agder, Grimstad 4879, Norway

CORRESPONDING AUTHOR: G. ACHALA (achalag.207ec011@nitk.edu.in)

ABSTRACT As data storage demands continue to grow, ensuring the reliability of NAND flash devices after a large number of Program/Erase (P/E) cycles have elapsed has become a critical requirement. With an addition in the number of Program/Erase (P/E) cycles in NAND flash devices, it is observed that there is an increase in the Raw Bit Error Rates (RBER) returned by the device. To address these challenges, this paper presents the design, implementation, and evaluation of advanced Error Correction Codes (ECC), including Euclidean Geometry Low-Density Parity Check (EG-LDPC), Quasi Cyclic Euclidean Geometry Low-Density Parity Check (QC-EG-LDPC), and Quasi-Cyclic Low-Density Parity Check (QC-LDPC) codes constructed over finite fields and adapted to meet the architectural constraints of flash memory devices. These codes are carefully optimized to meet the architectural and performance constraints of modern flash memory systems, with all designs achieving code rates in excess of 0.9. We have also synthesized LDPC codes having a code rate exceeding 0.92—to accommodate scenarios in which the parity check overhead has to be reduced to a minimum. The performance of the synthesized codes measured in terms of Undetected Bit Error Rate (UBER) has been evaluated as a function of the P/E cycles experienced by the NAND flash device. The process of constructing suitable parity check and generator matrices for each synthesized LDPC code is described. The designed QC-LPDC codes have been decoded using the Min Sum Algorithm (MSA) with a maximum of five decoding iterations. The evaluation of these codes has been compared with other state-of-the-art codes synthesized for this purpose, and an appropriate performance comparison has been presented. The encoders and decoders of the codes synthesized in this paper have been implemented using Xilinx Vivado software. It has been demonstrated that the QC-EG-LDPC code with a code rate of 0.92, synthesized in this paper, improves the lifetime of the NAND flash device to as many as 10,000 P/E cycles.

INDEX TERMS Error control code, memory device, NAND flash, LDPC codes, QC-LDPC, EG-LDPC, P/E cycles, MSA, encoder, decoder.

I. INTRODUCTION

NAND Flash memory has become an essential component of modern data storage solutions. It has revolutionized the non-volatile memory industry by offering high storage capacities, fast data access speeds, low latency during read operations, energy-efficient performance, and improved reliability [1], [2], [3]. Traditionally, the building block of NAND Flash memories has been the Single-Level Cell (SLC), where each memory cell accommodates just a bit of information, either a binary 0 or a binary 1.

In order to reach the ever-increasing requirements for greater storage density and to reduce the cost per bit, manufacturers have scaled down the physical size of flash memory and developed technologies capable of storing multiple bits per cell. This has led to the introduction of Multi-Level Cell (MLC) NAND flash, which is capable of holding two bits in one cell, effectively doubling the storage capacity of SLC. Further research and development have led to the design and realization of Triple-Level Cell (TLC) technology, Quad-Level Cell (QLC) technology, and Penta-Level Cell (PLC)

technology, which can respectively accommodate three bits, four bits, and five bits of data per cell [1].

These multi-bit storage techniques have significantly increased the data density and reduced storage costs. However, they also present added architectural and operational challenges. As more bits are stored in a single cell, the precision required to distinguish between the multiple voltage levels increases, making the memory highly susceptible to noise and interference. The reduced separation in voltage between two adjacent data representations gives rise to data recognition and retention issues. Moreover, such cells tend to have slower NAND read operations and reduced endurance, which requires the use of powerful error correction algorithms and wear-leveling techniques to maintain performance and reliability. Despite these challenges, the evolution from SLC to PLC has enabled flash memory to be applicable in diverse areas, spanning consumer electronics, data center storage, and enterprise systems [4], [5].

The Raw Bit Error Rate (RBER) refers to the rate at which bit errors occur in a memory device prior to any form of error detection or correction technique being applied [5], [6]. As flash memory has evolved from SLC to more advanced Multi-Level Cell architecture (MLC, TLC, QLC, and PLC), the RBER values have increased significantly. This is primarily attributed to the narrowing of voltage margins between neighbouring states. As the number of bits stored per cell increases, the sensitivity to noise, interference, and read/write disturbances also grows, thereby increasing the likelihood of bit errors, consequently leading to a higher RBER..

To effectively utilise these high-density memory technologies, it becomes essential to design and implement robust error control mechanisms, matched to the device architecture, capable of correcting as many raw bit errors as possible. The success of MLC, TLC, QLC, and PLC-based storage is highly dependent upon the ability of Error Control Coding (ECC) algorithms to reliably detect and correct errors and reduce the RBER values to levels that meet performance and reliability standards expected in typical applications.

Therefore, the development of powerful error detection and correction techniques has become a key area of focus in memory system design. These techniques must be capable of handling the increased error rates associated with multi-level cells without significantly compromising speed, energy efficiency, or storage overhead. Our research, driven by this need, has been directed towards the design and evaluation of powerful ECC schemes that can effectively lower the UBER returned by modern NAND flash devices to Bit Error Rate (BER) levels that are acceptable for real-world storage applications. By achieving this, we can enhance the overall reliability, endurance, and lifespan of next-generation non-volatile memory systems [1], [7].

The Undetected Bit Error Rate (UBER) represents the fraction of bit errors that persist even after applying error correction mechanisms, during data retrieval from a storage device [4]. Essentially, UBER reflects the residual error probability that is not identified or corrected by the error correction system, and it serves as a key indicator of the overall data integrity in the storage system.

An effective channel code significantly reduces the RBER, often lowering the error rate by several orders of magnitude. While RBER gives a measure of the initial noise and error characteristics of the memory hardware, UBER highlights how well the applied error correction technique performs in mitigating those errors. A low UBER is critical in ensuring the reliability and robustness of data, especially in high-storage capacity NAND flash architectures, where the risk of bit errors is substantially higher due to closely packed voltage states and increased susceptibility to interference.

In essence, the channel coding primarily aims to bring the UBER down to an acceptable level for consumer electronics as well as enterprise-grade systems, essentially ensuring that the storage device delivers highly reliable performance over its operational lifetime. The design of such powerful error control codes is, therefore, a fundamental requirement for the practical deployment of modern high-capacity non-volatile memory technologies.

A. RELATED WORK

In the early generations of flash memory, especially SLC NAND, Hamming codes were employed. Hamming codes offer single-error correction and double-error detection (SEC-DED) with low complexity, making them suitable for low-error-rate environments. However, they are inadequate for modern flash memory, where error rates are substantially higher. As error correction requirements grew, Bose–Chaudhuri–Hocquenghem (BCH) codes became the standard for MLC NAND flash. For a sector in NAND flash device capable of storing 4096 data bits and 128 parity bits, a BCH code was designed with the ability to correct at most 9 bits in error over a span of one sector (4096) information bits [5], [8]. BCH codes struggle to handle burst errors and exhibit increased latency for as error correction capabilities are scaled higher and higher. Reed-Solomon (RS) codes employ non-binary symbols for encoding and decoding and are capable of correcting burst errors. However, the non-binary nature of code increases the computational complexity while encoding and decoding.

In our prior research work, we have designed Subfield-Subcodes of Reed-Solomon (SSRS) codes matched to the architectures of MLC, TLC, QLC and PLC NAND flash devices. These codes were designed in such a manner that the number of voltage levels in NAND flash memory cell is equal to the number of elements in the chosen subfield of the SSRS code. For example, an SSRS code in which the operating subfield is chosen to be $GF(2^2)$, has been synthesized to handle error detection and correction in MLC flash memories (which are characterized by being able to store two bits per cell). Similarly, in SSRS codes capable of handling errors in TLC, QLC, and PLC memory devices, the operating subfields are $GF(2^3)$ for TLC, $GF(2^4)$ for QLC, and $GF(2^5)$ for PLC [4]. The key advantage of these code

is that they are very well matched to MLC, TLC, QLC and PLC flash memory architectures and also provide good error correction capability. These improvements come at the cost of higher encoding and decoding complexity. The use of these codes requires the use of an additional error magnitude calculation block after the deployment of Berlekamp-Massey and Chein search blocks to complete the process of decoding [4], [7], [9]. LDPC (Low-Density Parity-Check) codes were originally presented by Gallager in the 1960s [10]. In the late 1990s, Mackay and Neal reignited interest in LDPC codes, and exhibited their potential to closely approach the theoretical limits of channel capacity [11]. Since then, many researchers have demonstrated the application of these codes in many communication as well as storage applications [12], [13]. New code constructions and performance evaluation of many LDPC codes in varied applications are available in the literature. Practical encoder/decoder designs, hardware implementation strategies and performance evaluation have been carried out for many LDPC codes in different applications. This has resulted in LDPC codes being adopted in several modern communication standards- both wired and wireless [13].

Designing LDPC codes for NAND flash memory presents unique challenges not encountered in traditional communication and storage systems. High efficiency (code rate) is a key requirement. This leads to the design and deployment of LDPC codes with relatively high code rates—typically greater than 0.9 [6]. The challenge is to extract high error correcting capabilities from these high-rate LDPC codes, given the various error-inducing mechanisms that exist in NAND Flash cells. Another challenge is to design the code in such a manner that it is compliant with the architecture of various multi-level cell memories.

To meet the demand for high reliability in modern MLC, TLC, QLC and PLC devices, several LDPC check codes have been synthesized. LDPC codes are capable of offering high error correction capability through iterative decoding, particularly when enhanced with soft-decision inputs [14]. A variety of LDPC codes designed for NAND flash memory have been described in literature. Many of these codes that have been employed in NAND flash have used iterative decoding algorithms. LDPC codes designed using Eucledian Geometry results in improved UBER compared to regular LDPC codes constructed using computer search [15]. In recent years, LDPC codes designed with Quasi-cyclic (QC) properties have attracted the interest of researchers desiring to improve the data-intgerity levels in NAND memory [12]. Similarly, Spatially Coupled (SC), Globally Coupled (GC), and Dispersed Array (DA)LDPC codes also seem to have great potential in emerging memory applications [16], [17]. In recent years, there has been much research work carried in designing a variety of LDPC codes, including QC-EG-LDPC codes, protograph-based LDPC codes, and GC- LDPC codes [18]. However, QC-LDPC codes designed from Euclidean geometry and finite fields offer higher error correction ability. This has motivated us to work on the design and synthesis of a variety of QC-LDPC codes for NAND flash applications and propose suitable encoder and decoder models on FPGA platforms [19]. We have employed the hard decision decoding method (Bit-Flipping algorithm and Majority Logic Decoding algorithm) to decode the EG-LDPC codes. In literature, the decoding of LDPC codes is predominantly performed using soft-decision decoding algorithms. Among the most widely adopted techniques are the Belief Propagation (BP) algorithm, the Sum-Product Algorithm (SPA), the Min-Sum Algorithm (MSA), its improved variant, the Normalized Min-Sum Algorithm (NMS), and the Turbo Decoding Message Passing (TDMP) algorithm [20], [21], [22], [23]. We have employed the Min Sum algorithm to decode the synthesized QC-LDPC codes explained in this paper.

Flash memory is hierarchically organized into planes, blocks, pages, and sectors. Each plane contains multiple blocks, each block consists of several pages, and each page is further divided into sectors for efficient data management and error correction [4]. The LDPC codes have been synthesized keeping in view the device architecture of the storage devices.

The key contributions of this paper include

- Synthesizing (Designing) EG-LDPC codes with a maximum of 10% redundancy and designing a decoder using the Bit-Flipping (BF) and Majority Logic Decoding (MLD) algorithms.
- Synthesizing QC-EG-LDPC codes from $EG^*(2, 2^6)$ and $EG^*(2, 2^7)$ and QC-LDPC codes from finite fields $GF(67)$ and $GF(73)$, matching to different memory architectures and designing appropriate decoders using the min sum algorithm.
- Performance analysis using Monte-Carlo simulation for all the synthesized codes in which the UBER values returned by these codes have been estimated as a function of RBER and with respect to different Program/Erase cycles. Estimation of UBER with respect to Program/ Erase cycles helps us to appreciate the useful life of the NAND Flash memories protected by these codes.
- Simulation and synthesis of encoders and decoders of the designed codes using Xilinx Vivado 2022.2 software.

In this paper, section II recaps the channel model for MLC device. Design of EG-LDPC codes and decoders for EG-LDPC codes are described in Section III. The Generator matrix construction for QC-LDPC codes and QC-LDPC code construction using Euclidean geometry and finite fields are described in Section IV. Section V describes the synthesis of EG-LDPC, QC-EG-LDPC, and QC-LDPC codes from finite fields, which are matched to NAND flash memory architecture. This section also includes the performance analysis of synthesized codes as a function of RBER and p/E cycles. Section VI concludes the paper by summarizing the results and commenting upon their significance.

II. CHANNEL MODEL FOR FLASH MEMORY

We have taken the example of an MLC device capable of storing two bits per cell and possessing four distinct voltage

levels represented as V_1, V_2, V_3, V_4 to illustrate the various concepts involved in modeling NAND Flash memory as a channel. The NAND flash memory channel is mainly affected by programming noise, random telegraph noise (RTN), data retention noise, and cell-to-cell interference (CCI) [24], [25]. We have considered these noise components in modeling the NAND flash memory channel. The threshold voltage in terms of these noise components can be formulated as,

$$V = V_o + [N_u + N_p + T_n + R_n] \tag{1}$$

where, V_o =[V_1or V_2 or V_3 or V_4] are the write/verify voltage levels corresponding to each state.

N_u and N_p= Programming noise components depending on state (erasing and programming operation).
T_n =Random telegraph noise component depending on wear out.
R_n =Data retention noise component depending on time and program/erase cycles.

The Gaussian distribution function for the programming noise can be used to simulate this noise component and is given by [26],

$$p_p(v) = \begin{cases} \dfrac{1}{\sqrt{2\pi\sigma_e^2}} e^{-\frac{v^2}{2\sigma_e^2}}, & \text{for } V_o \in \{V_1\} \\ \dfrac{1}{\sqrt{2\pi\sigma_p^2}} e^{-\frac{v^2}{2\sigma_p^2}}, & \text{for } V_o \in \{V_{s2\sim4}\} \end{cases} \tag{2}$$

where, σ_e^2 and σ_p^2 are erase state and program state noise variances.

Repeated PE cycles in NAND flash memory harm the oxide layer of the cell's Floating Gate Transistor (FGT) by trap generation in the oxide. These affect the written voltage state, and errors induced due to these effects are called Random Telegraph Noise (RTN) induced errors. RTN noise is modeled by its probability distribution function (PDF) which can be described as [26],

$$p_{T_n}(v) = \frac{1}{\sqrt{2\pi\sigma_t^2}} e^{-\frac{v^2}{2\sigma_t^2}} \tag{3}$$

where, σ_t is the standard deviation with respect to RTN noise. The continuous charge leakage via the cell's FGT with advancing flash memory age is modeled using data retention noise R_n.

Retention noise is modeled as a Gaussian distribution [26]. The pdf is described by,

$$p_{R_n}(v) = \frac{1}{\sqrt{2\pi\sigma_r^2}} e^{-\frac{(v-\mu_R)^2}{2\sigma_R^2}} \tag{4}$$

where mean (μ_R) and standard deviation (σ_R) are given as,

$$\begin{aligned} \mu_R &= (V_o - x_0) \cdot \left[A_t (PE)^{\alpha_i} + B_t (PE)^{\alpha_o}\right] \cdot \log(1 + R_T) \\ \sigma_R &= 0.3\,|\mu_R| \end{aligned} \tag{5}$$

where x_0, A_t, B_t α_i, α_0 are constant parameters. V_o is the voltage state value. R_T is retention time (in years). For the calculation of the channel matrix, we have taken flash memory parameters from [24] and [26]. The parameter values are given as, $V_1 = 1.4, V_2 = 2.6$, $V_3 = 3.2$, $V_4 = 3.93$, $\Delta V_{pp} = 0.2$, $\sigma_e = 0.35$, $\sigma_p = 0.05$, $x_0 = 1.4$, $A_t = 0.000035$, $B_t = 0.000235$, $\alpha_i = 0.62$, and $\alpha_o = 0.3$, $R_T = 1$ year. The example Channel matrix derived for MLC NAND flash memory with P/E cycles of 100, 5000, 10000 is given in 6 7 and 8 respectively.

III. DESIGN OF EG-LDPC CODES

Cyclic LDPC codes are constructed based on the lines of the two-dimensional Euclidean geometry $EG(m, q) = EG(2, q)$ over $GF(q^s)$ with $q = 2^s$. A line in the Euclidean geometry $EG(m, q)$ can be realized in the field $GF(2^{s^2})$ as an extension field of $GF(2^s)$. Let α be a primitive element in $GF(2^{s^2})$. The points in $EG(2, 2^s)$ are $0, 1, \alpha, \alpha^2, \cdots, \alpha^{2s-2}$. Hence the

At P/E=100

$$P = \begin{bmatrix} 0.9982 & 0.001774 & 9.1213 \times 10^{-6} & 3.102 \times 10^{-10} \\ 1.43 \times 10^{-8} & 0.99996938 & 3.06 \times 10^{-5} & 5.654 \times 10^{-67} \\ 9.58 \times 10^{-69} & 1.13 \times 10^{-15} & 0.999999946 & 5.3382 \times 10^{-8} \\ 5.27 \times 10^{-225} & 7.27 \times 10^{-112} & 1.72 \times 10^{-20} & 1 \end{bmatrix} \tag{6}$$

At P/E=5000

$$P = \begin{bmatrix} 0.9980 & 0.0020 & 1.13 \times 10^{-5} & 4.2022 \times 10^{-10} \\ 9.742 \times 10^{-5} & 0.9977 & 0.0022 & 2.5141 \times 10^{-33} \\ 6.994 \times 10^{-33} & 5.46 \times 10^{-8} & 0.9999 & 7.2481 \times 10^{-5} \\ 1.43 \times 10^{-105} & 1.2431 \times 10^{-52} & 4.547 \times 10^{-10} & 0.99 \end{bmatrix} \tag{7}$$

At P/E=10000

$$P = \begin{bmatrix} 0.9977 & 0.00225 & 1.496 \times 10^{-5} & 8.487 \times 10^{-10} \\ 0.00254 & 0.98403 & 0.013422 & 2.55 \times 10^{-20} \\ 1.13 \times 10^{-19} & 3.313 \times 10^{-5} & 0.9984 & 0.0016 \\ 1.32 \times 10^{-61} & 5.4462 \times 10^{-31} & 2.2662 \times 10^{-6} & 0.99 \end{bmatrix} \tag{8}$$

Euclidean geometry $EG(2, 2^s)$ consists of 2^{2s} points and $2^s(2^s + 1)$ lines and each line has 2^s points. Each line in $EG(2, 2^s)$ is intersected by $2^s + 1$ points [19], [27].

Let $EG^*(2, q)$ be the subgeometry obtained by removing the origin of $EG(2, q)$ and the lines passing through the origin. Hence $EG^*(2, q)$ has $2^{2s} - 1$ points and $2^{2s} - 1$ lines. The minimum distance of a cyclic EG-LDPC code is $2^s + 1$. From the cyclic structure of $EG^*(2, 2^s)$, we can construct the parity check matrix H_{EG} of size $(q^2 - 1) \times (q^2 - 1)$. This is referred to as line point incident matrix. Let L_0 be the line in $EG^*(2, 2^s)$. H_{EG} matrix has $2^{2s} - 2$ rows corresponding to $L_0,\ \alpha L_0,\ \alpha^2 L_0,\ \cdots,\ \alpha^{2^{2s}-2} L_0$ lines. The rank of this matrix is $3^s - 1$. Null space over $GF(2)$ of H_{EG} results in EG-LDPC code with parameters $n = 2^{2s} - 1,\ k = 2^{2s} - 3^s$.

A. DECODERS FOR EG-LDPC CODESS

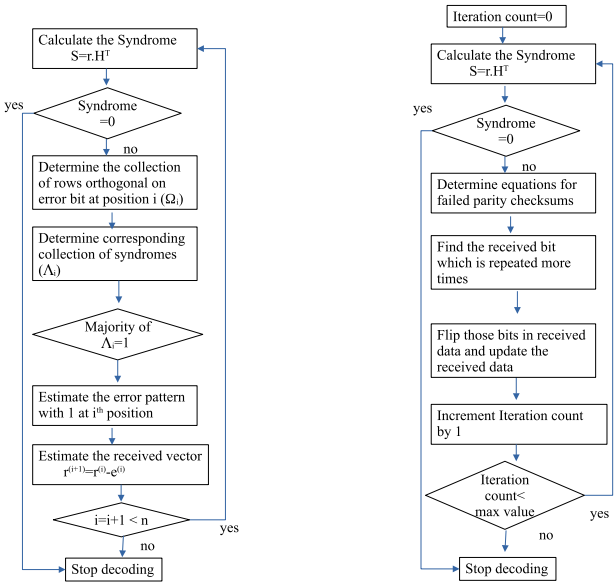
1) MAJORITY LOGIC DECODING (MLD)

In 1954, Reed introduced majority logic decoding of block codes by developing a decoding algorithm for the class of codes discovered by Muller [28], [29]. In 1961 Mitchel developed a similar decoding technique to decode cyclic Hamming codes. The cyclic codes that can be decoded efficiently using MLD were studied in 1967 by Rudolph [28]. MLD is relatively simple to implement, especially in hardware, due to its repetitive and logical nature.

Algorithm 1 Majority Logic Decoding (MLD) Algorithm for LDPC Decoding

Input: r, H
$\Omega \leftarrow H'$
for $i = 1$ to length(r) **do**
  $e \leftarrow \mathbf{0}$
  $syndrome \leftarrow rH'$
  $\Lambda \leftarrow syndrome \cdot \Omega(i, :)$
  if non-zero elements of $\Lambda > \frac{\text{non-zero elements of } \Omega(i,:)}{2}$ **then**
    $e(i) \leftarrow 1$
    $r \leftarrow (r + e)$
  end if
end for
Output: r

Consider $(n,\ k)$ cyclic code C and let v be a codeword in C that is written into a NAND flash device. Let $e = (e_0,\ e_1, \cdots,\ e_{n-1})$ be the error vector added to the stored data in the flash device due to the presence of noise. The corrupted data read from device be $r = (r_0, r_1, \cdots, r_{n-1})$. Parity check matrix $[H]_{M\times n}$ has row weight ρ and column weight γ. Based on the orthogonal structure of H, a simple decoding method that can correct up to $\lfloor \frac{\gamma}{2} \rfloor$ errors can be devised. Each row of the H matrix has a ρ number of ones, and these ρ positions will be significant in calculating the syndrome. The syndrome is calculated using 9. Equation 9 forms the M checksum equations. k^{th} checksum is essentially associated with k^{th} column of H^T is equivalently associated with k^{th} row of H. In the next step, find ω_i, which is a a collection of rows of H matrix which are orthogonal on every



(a) MLD (b) Bit Flipping

FIGURE 1. Flowcharts for MLD and BF decoding algorithms for designed EG-LDPC codes.

i^{th} position of vector e where $0 \le I \le (n - 1)$. Each ω_i has γ numbers of entries.

$$S = r.H^T = (v + e).H^T = e.H^T \tag{9}$$

From ω_i, the corresponding collection of syndrome will result in Λ_i. If the majority of vector Λ_i is equal to one, then the corresponding error bit position is made one ($e = [e_0,\ e_1, \cdots,\ e_{i-1},\ e_i = 1,\ e_{i+1},\ \cdots,\ e_{n-1}]$). The received data r is updated by subtracting vector e from r. Again, the syndrome is calculated for updated r, and if the syndrome becomes zero, the decoding process stops. If the syndrome is not a zero, the process continues until the majority checks for the error bit at $(n-1)$ position $(i = (n-1))$. The flow chart for MLD is depicted in Fig. 1. The algorithm is presented below. Here. represents the mod 2 product.

2) BIT FLIPPING ALGORITHM

Bit Flipping algorithm for LDPC codes was introduced by Gallager in 1962 [10]. Along with defining LDPC codes, Gallager proposed the bit-flipping algorithm as one of the first decoding methods for these codes. His decoding strategy was motivated by the simplicity of operations, using basic bit manipulations rather than complex probabilistic models. It computes the syndrome, which is the result of multiplying the received word by the parity-check matrix. A zero syndrome indicates no errors, while a non-zero syndrome implies errors. For each bit, it checks how many of the parity checks it is involved in are unsatisfied. If a received bit (error bit) is repeated more times in failed check-sums, or a received bit (error bit) that is repeated more than a threshold value is flipped. The received message is updated and checked for syndrome again. If the syndrome is non-zero, the same

Algorithm 2 Bit-Flipping Algorithm for LDPC Decoding

Input: r, H, $Iter$ ▷r: Received vector, H: Parity-check matrix, $Iter$: Maximum iterations
for $ite = 1$ to $Iter$ **do**
  $rx \leftarrow r$
  $syndrome \leftarrow rH'$
  if $syndrome == 0$ **then**
    print "Decoding successful in $ite - 1$ iterations"
    break
  end if
  $rep\_vector \leftarrow syndrome \cdot H$
  $max\_positions \leftarrow$ find($rep\_vector == \max(rep\_vector)$)
  for $i = 1$ to length($max\_positions$) **do**
    $r(max\_positions(i)) \leftarrow \complement r(max\_positions(i))$
  end for
  if $rx == r$ **then**
    print "Repetition of rx vector, stop decoding"
    break
  end if
end for
return r

procedure is repeated iteratively until the maximum number of iteration values is reached [19]. The Bit-Flipping decoding method is presented in Algorithm 2, and the flowchart is depicted in Fig. 1b.

IV. QC-LDPC CODES

Quasi-Cyclic codes are a subclass of linear block codes that do not possess a fully cyclic structure. A (n, k) QC-LDPC code has a code word of length $n = t_{qc}b$ with t_{qc} sections, where each section has b bits. The length of information $k = c_{qc}b$, where c_{qc} is an integer, $1 \leq c_{qc} \leq t_{qc}$, and each section c_{qc} has b bits. The systematic generator matrix $G_{qc_{sys}}$ is a $t_{qc} \times c_{qc}$ array of circulants. Each circulant is of size $b \times b$. The parity check matrix $H_{qc_{sys}}$ has $(t_{qc} - c_{qc}) \times t_{qc}$ an array of elements [19], [27].

A. QC-LDPC CODE ENCODER

The parity check matrix H_{qc} constructed using quasi-cyclic format results in a non-systematic format. Hence the obtaining the generator matrix G_{qc} from H_{qc} is not straightforward [19].

1) FULL RANK H MATRIX

If the parity check matrix is a full rank matrix, then the rank of the matrix is equal to $(t_{qc} - c_{qc})b$. Choose another

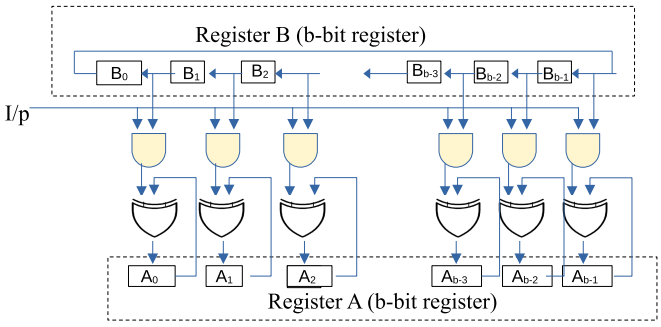


FIGURE 2. Generalized encoder circuit for QC-LDPC codes.

matrix D of size $(t_{qc} - c_{qc})b \times (t_{qc} - c_{qc})b$ from H_{qc} which also has the same rank as H_qc. Using matrix D obtain the parity matrix P to form the $G_{qc,sys} = [PI]$ in systematic form. The procedure followed to obtain matrix $G_{qc,sys}$ is briefly explained in Algorithm 3. The generalized generator matrix QC code is in (10). as shown at the bottom of the page.

2) NON FULL RANK H MATRIX

If the rank of parity check matrix H_{qc} is $r \neq (t_{qc} - c_{qc})$ then the generator matrix $G_{qc,semi-sys}$ will have the semisystematic circulant format. The $G_{qc,semi-sys}$ is composed of two submatrices Q and $G_{qc,sys}$. First identify the l $((t_{qc} - c_{qc}) \leq l \leq t_{qc})$ column blcok in H_{qc} to form matrix D which has rank r. Follow the procedure mentioned in Algorithm 3 to obtain $G_{qc,sys}$. The size of the matrix $G_{qc,sys}$ is $(t_{qc} - l)b \times t_{qc}b$ which has systematic circulant format. Procedure to obtain $Q_{(lb-r)\times t_{qc}b}$ is elaborated in Algorithm 4. The generator matrix $G_{qc,semi-sys}$ in semisystematic form is given below.

$$G_{qc,semi-sys} = \begin{bmatrix} Q \\ G_{qc,sys} \end{bmatrix} \tag{11}$$

B. ENCODER CIRCUIT FOR QC-LDPC CODES

The encoder for QC-LDPC codes is designed using a Cyclic Shift-Register-Adder-Accumulator (CSRAA) circuit depicted in Fig. 2 [19], [27]. Each CSRAA circuit contains two $b = (q - 1)$ bit registers. Among them, B is a feedback shift register and A is an accumulator. Encoded data $v = (p_0, p_1, \cdots, p_{t_{qc}-c_{qc}} : u_0, u_1, \cdots, u_{c_{qc}} - 1)$ is computed using the Generator matrix mentioned in (10). Here, each parity symbol p_i and message symbol u_j is $b = (q - 1)$ bit wide. Hence $c_{qc} \times b = k$ and $t_{qc} \times b = n$. Each parity symbol is formulated by $p_i = u_0 G_{0,i} + u_1 G_{1,i} + \cdots + u_{c_{qc}-1} G_{c_{qc}-1,i}$ intern each $u_j \times G_{j,i} = u_{jb} g^{(0)}_{j,i} + u_{jb+1} g^{(1)}_{j,i} + \cdots + u_{(jb+b-1)} g^{(b-1)}_{j,i}$. In CSRAA circuit the $g^{(b-1)}_{(c_{qc}-1),i}$ is loaded in to register, B and content of register A is set to zero. The message pin

$$\mathbf{G}_{qc_{qc},s\ y_s} = \begin{bmatrix} \mathbf{G_0} \\ \mathbf{G}_1 \\ \vdots \\ \mathbf{G}_{c_{qc}-1} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{G}_{0,0} & \mathbf{G}_{0,1} & \cdots & \mathbf{G}_{0,t_{qc}-c_{qc}-1} & \mathbf{I}\ \mathbf{O} \cdots \mathbf{O} \\ \mathbf{G}_{1,0} & \mathbf{G}_{1,1} & \cdots & \mathbf{G}_{1,t_{qc}-c_{qc}-1} & \mathbf{O}\ \mathbf{I} \cdots \mathbf{O} \\ \vdots & \vdots & \ddots & \vdots & \vdots\ \vdots\ \ddots\ \vdots \\ \mathbf{G}_{c_{qc}-1,0} & \mathbf{G}_{c_{qc}-1,1} & \cdots & \mathbf{G}_{c_{qc}-1,t_{qc}-c_{qc}-1} & \mathbf{O}\ \mathbf{O} \cdots \mathbf{I} \end{bmatrix}}_{\mathbf{P}} \tag{10}$$

Algorithm 3 QC- LDPC Generator Matrix Construction in Systematic Form

Input: H_{qc}: Quasi-cyclic parity-check matrix, n: Codeword length, k: Message length, t_{qc}: Number of column blocks, c_{qc}: is chosen such that $(t_{qc}-c_{qc})$ Number of row blocks in H_{qc}, b: CPM size
Output: Generator matrix $G_{qc,sys}$
Initialize matrices $P \in \mathbb{F}_2^{k\times(n-k)}$ and $G \in \mathbb{F}_2^{k\times n}$ as zero matrices.
Extract matrix $D \leftarrow (t_{qc}-c_{qc}) \times (t_{qc}-c_{qc})$ block from H_{qc}
Rank Check for M **and** D
if Rank(H_{qc}) == Rank(D) **then**
Initialize matrix z_i = generator g_i = $(g_{i,0},\ g_{i,1},\ \cdots,\ g_{i,t_{qc}-c_{qc}-1})$
  Compute the inverse of D: D_{inverse}
  Initialize $x = (1,\ 0, \cdots,\ 0)$ is a b-tuple
  for $i = 0$ to $c_{qc}-1$ **do**
    $M_{t_{qc}-c_{qc}+i} \leftarrow (t_{qc}-c_{qc}+i)^{th}$ column of block H_{qc}
    $z_i^T = D^{-1}M_{t_{qc}-c_{qc}+i}x^T$
  end for
  First row of G is $g_{i,j} \leftarrow z_0,\ z_1,\ \cdots,\ z_{c_{qc}-1}$
  Parity matrix $P \leftarrow (b \times b)$ circulants from $g_{i,j}$
  Form generator matrix $G \leftarrow [P\ I_k]$ where I_k is the $k \times k$ identity matrix
end if
Verification
Compute check $\leftarrow$ $G_{qc,sys} \cdot (H_{qc}^T)$, check=0, indicates generated matrix is valid.

Algorithm 4 Q Matrix Construction in Semisystematic Format

Input: H_{qc}: Quasi-cyclic parity-check matrix, $D \leftarrow$ Submatrix extracted from H_{qc}, Parameters: c_{qc}, b, t_{qc}, l, $[G_{sys}]_{(t_{qc}-l)\times t_{qc}}$ computed using Algorithm 3.
Output: Semi-systematic generator matrix $G_{qc\_semisys}$
Generate Q **Matrix**
Define d: Column block-wise distribution vector of dependency columns
Determine dependent columns in D: dependentCols_D
Define $w_{\text{nnz}(d)\times l\cdot b}$, initialized as 0
Define $Q_{\text{matrix}} \leftarrow \mathbf{0}_{\sum d\times t_{qc}\cdot b}$
Identify the block position of dependency columns in d: positions $\leftarrow$ find(d)
for $j = 1$ to nnz(d) **do**
  Determine position: pos $\leftarrow$ positions(j)
  Determine number of dependency columns: dep $\leftarrow$ d(pos)
  Extract known indices: known_indices $\leftarrow$ dependentCols_D($(j-1)\cdot$dep$+1 : j\cdot$dep)
  Assign $w(j,$known_indices$) \leftarrow [1,0,\ldots,0]$
  Solve for $w_i \leftarrow$ Solve for $D.w_i^T = 0$
  Compute $Q_i \leftarrow$ Block-wise circular shift of(w_i) $(b-1)$ times
  Update $Q_{\text{matrix}}((j-1)\cdot\text{dep}+1 : j\cdot\text{dep}, 1 : \text{size}(Q_i,2)) \leftarrow Q_i$
end for
Form Final Semi-Systematic Generator Matrix
$G_{qc\_semisys} \leftarrow [Q_{\text{matrix}}; G_{\text{sys}}]$ (vertical padding)
Verification
Compute check $\leftarrow$ nnz$G_{qc_semisys} \cdot (H_{qc}^T)$ check=0, indicates generated matrix is valid.

u_j is loaded with the last bit $u_{c_{qc}b-1}$. Then the multiplication operation $u_{c_{qc}b-1} \times g^{(b-1)}_{(c_{qc}-1),i}$is performed using b number of AND gates and added to the content in the register A using XOR gates. The result is accumulated in A. In the next step, the content of register B is shifted left by one time, and the message pin is loaded with the next message bit. This shift-multiply-add and accumulate operation is continued until the $u_{(c_{qc}-1)b}$ message is loaded into the message pin and we obtain the result of $u_(c_{qc}-1)G_{(c_{qc}-1),i}$. In the next step register B is loaded with $g^{(b-1)}_{(c_{qc}-2),i}$ and above mentioned procedure is repeated to compute $u_(c_{qc}-2)G_{c_{qc}-2,i}$. Note that each time the accumulator register A will not be refreshed. The procedure is repeated until the accumulator has the result of $p_i = u_0G_{0,i} + u_1G_{1,i} + \cdots + u_{c_{qc}-1}G_{c_{qc}-1,i}$. Similarly, different parity check symbols are calculated in parallel using similar CSRAA circuits.

C. DECODING ALGORITHM FOR QC-LDPC CODES

Efficient decoding methods for sparse graph-based codes, including message-passing algorithms like Belief Propagation (BP) and its approximations, were first introduced by Mackay in 1999 [11]. A popular iterative message-passing algorithm for decoding error-correcting codes, like Low-Density Parity-Check (LDPC) codes, is the Min-Sum Algorithm (MSA) and Sum Product Algorithm(SPA) [14]. It is a simpler version of the BP method, which approximates exponential and logarithmic operations to lower computer complexity. In MSA, based on the received data, a log-likelihood ratio (LLR) is initialised for each variable node. Each variable node sends a message to connected check nodes, representing its belief about the bit's likelihood. Each check node calculates a message for connected variable nodes based on the parity-check constraints. The above procedure to updating variable nodes and check nodes is repeated for a fixed number of iterations. The estimated data is finally obtained based on the variable node updates. The Normalized Min Sum (NMS) algorithm further improves MSA by introducing a normalization factor to scale messages and reduce overestimation errors.

Let H be the parity check matrix of $(n,\ k)$ LDPC codes with n columns and m rows. Let $N(k)$ represent locations of 1's in the k^{th} row and $M(l)$ represent the 1's in the l^{th} column of H. The message bit passing from Variable Node (VN) to Check Node (CN) is $L_{k\to l}$ set to LLR values if the element $h_{k,l}$ in H matrix is equal to 1. For $0 \le k < m$ and $0 \le l < n$, the CN is updated using (12).

$$L_{k\to l} = \prod_{l'\in N(k)\setminus(l)} \alpha_{l'i} \min_{l'\in N(k)\setminus\{l\}} \beta_{l'k} \tag{12}$$

Algorithm 5 Min-Sum Decoder

Input: LLR: Log-Likelihood Ratios (LLRs) of received data,
H: Parity-check matrix,
$maxIterations$: Maximum number of iterations,

Output: LLR_{final}: Updated LLR values
Initialize dimensions of H: $[m, n] \leftarrow \text{size}(H)$
Initialize check-to-variable node messages: $C_v \leftarrow \mathbf{0}_{m \times n}$
Initialize variable-to-check node messages: $V_c \leftarrow \text{repmat}(LLR^{\top}, m, 1)$
for iter = 1 to $maxIterations$ **do**
  Update check-to-variable node messages (C_v):
  for each check node $k \in \{1, 2, \ldots, m\}$ **do**
    Find variables connected to k: connectedVars $\leftarrow \{l \mid H(k, l) = 1\}$
    for each variable $l \in$ connectedVars **do**
      Other variables connected to k: ExtraVars $\leftarrow$ connectedVars $\setminus \{l\}$
      Update $C_v(k, l)$:

$$C_v(k, l) \leftarrow \prod_{s \in \text{ExtraVars}} \text{sign}(V_c(k, s)) \cdot \min_{k \in \text{ExtraVars}} |V_c(k, s)|$$

    end for
  end for
  Update variable-to-check node messages (V_c):
  for each variable node $l \in \{1, 2, \ldots, n\}$ **do**
    Find check nodes connected to l: connectedChecks $\leftarrow \{k \mid H(k, l) = 1\}$
    for each check node $k \in$ connectedChecks **do**
      Other checks connected to l: ExtraChecks $\leftarrow$ connectedChecks $\setminus \{k\}$
      Update $V_c(k, l)$:

$$V_c(k, l) \leftarrow LLR(l) + \sum_{k \in \text{ExtraChecks}} C_v(k, l)$$

    end for
  end for
  Compute final LLR values:

$$LLR_{\text{final}} \leftarrow LLR + \sum_{k=1}^{m} C_v(k, :)$$

end for

Here, $L_{k \to l} = \alpha_{kl}\beta_{kl}$. That is α_{kl} is sign of $L_{k \to l}$ and β_{kl} is the magnitude of $L_{k \to l}$. The scaled or normalized MSA is obtained by multiplying CN update equation (12) by a factor $a_{atten}(0 < a_{atten} < 1)$ which inturn enhances the performance of decoding algorithm [19]. Detailed steps involved in MSA is explained in Algorithm 5.

D. CONSTRUCTION OF QC-LDPC CODES

1) QC-EG-LDPC CODES

Consider the $H_{EG,cyc}$ matrix constructed using $EG^*(2, q)$, where each line has q points. The size of $H_{EG,cyc}$ is

Algorithm 6 QC-EG-LDPC Parity Check Matrix Generation

Input: s: Base size parameter of $EG^*(2, 2^s)$
Output: H_{qc}: Quasi-cyclic parity-check matrix
Compute $q \leftarrow 2^s$
Define line h of size $(1, q^2 - 1)$: line not passing through origin in $EG^*(2, 2^s)$
Initialize permutation matrix π_{row} of size $(q+1) \times (q-1)$
Step 1: Generate Permutation Indices
for $k = 1$ to $q + 1$ **do**
  for $j = 1$ to $q - 1$ **do**
    $\pi_{\text{row}_m}(k, j) \leftarrow ((j - 1) \cdot (q + 1)) + k$
  end for
end for
Flatten π_{row_m} row-wise: $\pi_{row,k} == \pi_{column,l}$

$$\pi_{\text{col}} \leftarrow \text{reshape}(\pi_{\text{row}_m}^{\top}, 1, [])$$

$$\pi_{\text{row}} \leftarrow \text{reshape}(\pi_{\text{row}_m}^{\top}, 1, [])$$

Step 2: Construct Initial Parity-Check Matrix
Initialize H_0 as a zero matrix of size $(q^2 - 1) \times (q^2 - 1)$
for $i = 1$ to length(π_{row}) **do**
  $H_0(i, :) \leftarrow \text{circshift}(h, [0, \pi_{\text{row}}(i) - 1])$
end for
Step 3: Generate QC Parity-Check Matrix

$$H_{qc} \leftarrow H_0(:, \pi_{\text{col}})$$

$(q^2 - 1) \times (q^2 - 1)$. Let $\pi_{row,k}$ and $\pi_{column,l}$ are the indices defined as in (13) where $(0 \le k, l \le q)$ [19].

$$\begin{aligned} &\pi_{row,k} \\ &= [k, (q+1) + k, 2(q+1) + k, \cdot, \ (q-2)(q+1) + k] \\ &\pi_{column,l} \\ &= [l, (q+1) + l, 2(q+1) + l, \cdot, \ (q-2)(q+1) + l] \end{aligned} \quad (13)$$

All the rows and columns of $H_{EG,cyc}(2, q)$ are defined by considering $\pi_{row,k}$ and $\pi_{column,l}$ as $\pi_{row} = [\pi_{row,0}, \pi_{row,1}, \cdots, \pi_{row,Q}]$ and $\pi_{column} = [\pi_{column,0}, \pi_{column,1}, \cdots, \pi_{column,Q}]$. Permutation of rows and columns of $H_{EG,cyc}(2, q)$ with respect to π_{row} and π_{column} results in a quasi-cyclic structure with array of size $(q+1) \times (q+1)$. Each array is having $(q-1)$ rows and $(q-1)$ columns. Detailed steps are explained in Algorithm 6. Let γ and ρ are the two positive integers with condition $1 \le \gamma \le q + 1$ and $1 \le \rho \le q + 1$. Matrix $M_{EG,qc}(\gamma, \rho)$ forms the matrix of size $\gamma(q-1) \times \rho(q-1)$. The null space generated by $M_{EG,qc}$ results in QC-EG-LDPC codes.

2) TYPE-1 QC-LDPC CODES FROM FINITE FIELDS

Type-1 QC-LDPC codes are constructed using finite fields (Galois Fields). The parity check matrix is constructed based on the finite field elements and will form the quasi-cyclic structure based on the Circulant Permutation Matrix (CPM) dispersion [19], [30]. The base matrix B generated from finite field elements has to satisfy the $2 \times 2/3 \times 3$ submatrix (SM)

constraints. This ensures the girth in the Tanner graph is at least 8 or more.

Consider finite field $GF(q)$ with primitive element α. The elements of $GF(q)$ are $\{0,\ 1,\ \alpha,\ \alpha^2,\ \cdots, \alpha^{q-2}\}$. Consider two arbitrary set of distinct elements $S_a,\ S_b$ from $GF(q)$ such that $S_a = \{\alpha^{a_0},\ \alpha^{a_1},\ \cdots,\ \alpha^{a_{k-1}}\}$ and $S_b = \{\alpha^{b_0},\ \alpha^{b_1},\ \cdots,\ \alpha^{b_{l-1}}\}$ for $1 \leq k,\ l \leq q$. Here $a_m,\ b_n$ are the powers of α. The base matrix is constructed using Eq. (14), as shown at the bottom of the page.

The base matrix presented in (14) ensures that the elements of the matrix are the linear sum of elements of two subsets. Each row has distinct elements and almost one zero element. Two rows in B do not have any identical element at any position. The $(q-1)$ fold CPM dispersion $(q-1 \times q-1)$ of all the elements of B generates the parity check matrix H.

The LDPC codes are constructed based on the Finite fields with $GF(2)$ and its powers, which put limitations on the code parameters. To design the codes with the desired length that match the given constraints, LDPC codes are constructed based on the prime fields other than 2.

V. DESIGN AND PERFORMANCE ANALYSIS OF DESIGNED LDPC CODES FOR NAND FLASH MEMORY APPLICATIONS

A. EG-LDPC CODES

NAND Flash memory applications require the code rate of the designed ECC to be as large as possible to keep the redundancy overhead to a minimum. Keeping this requirement in mind, we have initiated our design of LDPC codes with a candidate from the Euclidean geometry $EG^*(2,\ 2^8)$. EG-LDPC codes generated from $EG^*(2,\ 2^6)$ have the parameters $n = 4095,\ k = 3367$ and a code rate $R = k/n$ of 0.822. Similarly, the (16383, 14197) EG-LDPC code generated from the Euclidean geometry $EG^*(2,\ 2^7)$ also has a code rate R less than 0.9. The line in $EG^*(2,\ 2^8)$ has 256 points. Repeated circular shifts of this line produce H, a matrix of size 65535×65535. In this case, the code parameters are length of the codeword $n = 65535$ and length of message sequence $k = 58975$. The corresponding rate is $R = 0.9$. The synthesised code is decoded using MLD and BF (5 iterations) decoding algorithms. The performance of this code when deployed in MLC NAND flash memory applications (using the channel model specified in Section II is plotted in Fig. 3. Since MLD approach is computationally complex, it was not possible to complete Monte Carlo simulation runs in our system with RAM size of 82 GB with Intel(R) Xeon(R) E-2224G CPU @ 3.50GHz processor. Therefore, we have used the BF decoding algorithm with ten iterations to determine Monte Carlo simulation results in MATLAB R2024 b. A study of the plot in Fig. 3 reveals that the obtained values

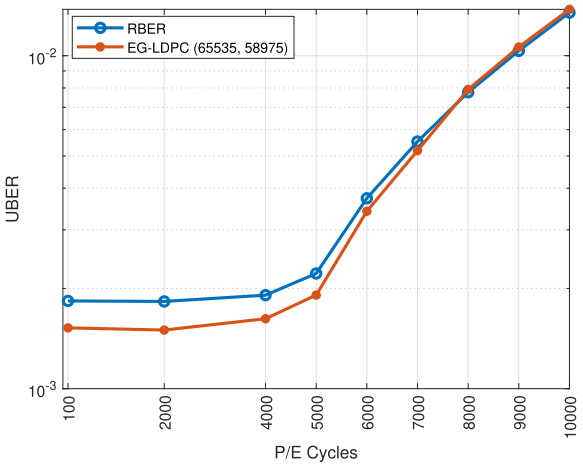


FIGURE 3. Performance analysis of EG-LDPC codes for different P/E cycles.

of UBER doesnot provide significant improvement compared to RBER values. Hence, we conclude that the designed EG-LDPC(65535, 58975) is not suitable for being deployed in NAND flash memory applications.

B. QC-EG-LDPC CODES

The NAND flash memory device is structured based on planes, blocks, and pages. Each page is divided into sectors to implement error control coding techniques. The information stored in a sector typically conforms to standardized sizes such as 512 bytes, 2 kilobytes (KB), or 4 kilobytes (KB), depending on the storage device architecture and applications [31]. Here, to design two-dimensional Euclidean geometry codes, we have considered a sector that is capable of holding 512 bytes (4096 bits) of data. Keeping this architectural constraint, we have designed QC-EG-LDPC codes from the Euclidean geometry $EG^*(2, 2^6)$. The Quasi-cyclic structured array of size 65×65 is generated from $EG^*(2, 2^6)$ with each array of size 63×63 using Algorithm 6. Hence, the original parity check matrix H_{qc} is of the size 4095×4095. The line considered here to synthesize parity check matrix is in Eq. (15), as shown at the bottom of the next page.

By choosing $\gamma = 4$ and $\rho = 65$, we get matrix $M_{EG,qc}$ of size 252×4095 where $M_{EG,qc} = H_{qc(1:252,1:4095)}$. The rank of this matrix is 240. This gives rise to a QC-EG-LDPC code with $n = 4095$, $k = 3855$, and $R = 0.94$. Since the matrix $M_{EG,qc}$ is not a full rank matrix, Algorithm 4 is followed to generate G, the generator matrix in semi-systematic format. The sub matrix $D = M_{EG,qc(1:252,1:252)}$ is choosen to generate the systematic part of G. The density of parity check matrix is 0.0156 with row weight of 64 and column weights of 3 and 4.

$$B(k,l) = \begin{bmatrix} \eta\alpha^{a_0} + \alpha^{b_0} & \eta\alpha^{a_0} + \alpha^{b_1} & \cdots & \eta\alpha^{a_0} + \alpha^{b_{l-1}} \\ \eta\alpha^{a_1} + \alpha^{b_0} & \eta\alpha^{a_1} + \alpha^{b_1} & \cdots & \eta\alpha^{a_1} + \alpha^{b_{l-1}} \\ \vdots & \vdots & \vdots & \vdots \\ \eta\alpha^{a_{k-1}} + \alpha^{b_0} & \eta\alpha^{a_{k-1}} + \alpha^{b_1} & \cdots & \eta\alpha^{a_{k-1}} + \alpha^{b_{l-1}} \end{bmatrix} \tag{14}$$

We have synthesized another LDPC code from $EG^*(2, 2^6)$ with parameters $n = 4095$ and $k = 3771$ with rate 0.92 by choosing $\gamma = 6$ and $\rho = 65$. Here $M_{EG,qc} = H_{qc(1:378,1:4095)}$ and $D = M_{EG,qc(1:378,1:378)}$. Since $M_{EG,qc}$ is not a full rank matrix, the generator matrix is formed using a semi-systematic form. Here the density of of parity check matrix is 0.0156 with row weight of 64 and column weight of 3 and 4. The QC-EG-LDPC codes from Eucledian Geometry $EG^*(2, 2^7)$ are designed for a sector of size 16383 bits [31], [32]. The Qussi cyclic structured array of size 129×129 is generated from $EG^*(2, 2^6)$ with each array of size 127×127. This results in (16383, 16383)H_{qc} matrix. The line from $EG^*(2, 2^7)$ used to obtain H_{qc} is in Eq. (16), as shown at the bottom of the page.

The matrix $M_{EG,qc}(508, 16383)$ with $\gamma = 4$, $\rho = 129$ has rank 494, where $M_{EG,qc} = H_{qc(1:508,1:16383)}$. Since the rank of $M_{EG,qc}$ is 494, null space over $GF(2)$ results in (16383, 15889) QC-EG-LDPC code. The parity check matrix has a density of 0.0078 with row weight of 128 and column weights of 3 and 4.

Two QC-EG-LDPC codes have been synthesized from $EG^*(2, 2^7)$ with a code rate $R = 0.907$ and $R = 0.97$.

$M_{EG,qc}(2540, 16383)$ with $\gamma = 20$, $\rho = 129$ has rank 1518 where $M_{EG,qc} = H_{qc(1:2540,1:16383)}$. The nullspace over $GF(2)$ results in (16383, 14865) QC-EG-LDPC code with rate of 0.90. Since $M_{EG,qc}(2540, 16383)$ is not a full rank matrix, a semisystematic approach is applied to obtain the Generator matrix. The density of parity check matrix is 0.0078.

The performance analysis of (4095, 3855), (4095, 3771), (16383, 15889), (16383, 14865) QC-EG-LDPC codes with repect differenct P/E cycles are plotted in Fig. 4. We have employed the Min Sum Algorithm (MSA) algorithm with five iterations for Monte-carlo simulations. The UBER plot for higher RBER values is also depicted in Fig. 5.

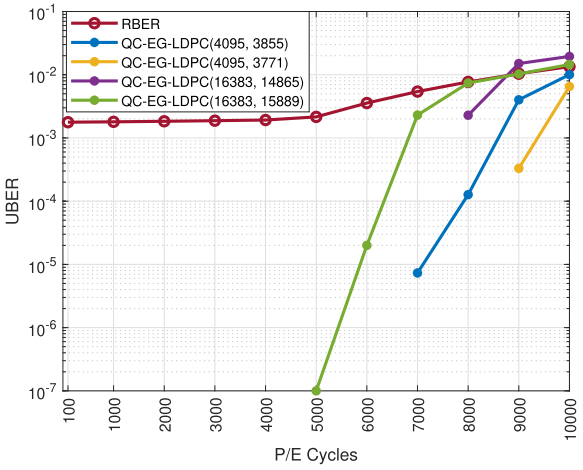


FIGURE 4. Performnce analysis of QC-EG-LDPC codes for different P/E cycles.

C. QC-LDPC CODES FROM FINITE FIELDS

The architecture of the 1GB NAND flash memory device designed by Micron Technology has been considered for the design of following LDPC code [33]. When the device is scaled from SLC to MLC, each page in MLC logically devided into Most significant Bit (MSB) page and Least Significant Bit (LSB) page. In this case, each MSB and LSB page has the capacity to store 2112 bytes of information. In order to design LDPC codes for this page, each page is divided into four sectors, where each sector can accommodate 528 bytes (4224 bits) of information.

Taking this architecture into consideration, we synthesized QC-LDPC codes over $GF(67)$ using the algorithm 7. All the elements of $GF(67)$ are generated using the primitive element $\alpha = 2$. For the design purpose the choosen elements of subset $S_1 = 1, \alpha, \alpha^2, \alpha^3$, and subset $S_2 = \alpha^2, \alpha^3, \cdots \alpha^{65}$. This

$$L_1 = \{\alpha, \alpha^7, \alpha^{37}, \alpha^{39}, \alpha^{50}, \alpha^{64}, \alpha^{84}, \alpha^{142}, \alpha^{215}, \alpha^{355}, \alpha^{439}, \alpha^{448}, \alpha^{612}, \alpha^{693}, \alpha^{759}, \alpha^{898}, \alpha^{1000}, \alpha^{1007}, \alpha^{1017}, \alpha^{1046}, \alpha^{1199}, \alpha^{1281}, \alpha^{1424}, \alpha^{1475}, \alpha^{1498}, \alpha^{1687}, \alpha^{1724}, \alpha^{1837}, \alpha^{2147}, \alpha^{2159}, \alpha^{2219}, \alpha^{2223}, \alpha^{2245}, \alpha^{2273}, \alpha^{2313}, \alpha^{2344}, \alpha^{2368}, \alpha^{2429}, \alpha^{2488}, \alpha^{2496}, \alpha^{2540}, \alpha^{2575}, \alpha^{2596}, \alpha^{2676}, \alpha^{2738}, \alpha^{2786}, \alpha^{2855}, \alpha^{2908}, \alpha^{3023}, \alpha^{3026}, \alpha^{3041}, \alpha^{3042}, \alpha^{3130}, \alpha^{3200}, \alpha^{3242}, \alpha^{3369}, \alpha^{3402}, \alpha^{3526}, \alpha^{3531}, \alpha^{3622}, \alpha^{3663}, \alpha^{3760}, \alpha^{3866}, \alpha^{3941}\} \quad (15)$$

$$L_2 = \{\alpha, \alpha^7, \alpha^{59}, \alpha^{128}, \alpha^{183}, \alpha^{537}, \alpha^{873}, \alpha^{896}, \alpha^{1180}, \alpha^{1194}, \alpha^{1233}, \alpha^{1390}, \alpha^{1394}, \alpha^{1522}, \alpha^{1559}, \alpha^{1760}, \alpha^{1969}, \alpha^{1982}, \alpha^{2013}, \alpha^{2357}, \alpha^{2623}, \alpha^{2840}, \alpha^{2852}, \alpha^{2956}, \alpha^{3094}, \alpha^{3204}, \alpha^{3305}, \alpha^{3491}, \alpha^{3539}, \alpha^{3593}, \alpha^{3912}, \alpha^{3955}, \alpha^{4507}, \alpha^{4584}, \alpha^{4630}, \alpha^{4947}, \alpha^{5198}, \alpha^{5226}, \alpha^{5304}, \alpha^{5351}, \alpha^{5385}, \alpha^{5618}, \alpha^{5626}, \alpha^{5882}, \alpha^{5956}, \alpha^{6095}, \alpha^{6287}, \alpha^{6358}, \alpha^{6503}, \alpha^{6773}, \alpha^{6776}, \alpha^{6802}, \alpha^{6864}, \alpha^{7041}, \alpha^{7209}, \alpha^{7389}, \alpha^{7552}, \alpha^{7757}, \alpha^{7779}, \alpha^{7951}, \alpha^{8084}, \alpha^{8425}, \alpha^{8518}, \alpha^{8542}, \alpha^{8569}, \alpha^{8750}, \alpha^{9026}, \alpha^{9246}, \alpha^{9448}, \alpha^{9465}, \alpha^{9820}, \alpha^{9916}, \alpha^{10024}, \alpha^{10159}, \alpha^{10293}, \alpha^{10377}, \alpha^{10467}, \alpha^{10651}, \alpha^{10662}, \alpha^{10748}, \alpha^{10985}, \alpha^{11057}, \alpha^{11505}, \alpha^{11852}, \alpha^{11919}, \alpha^{11961}, \alpha^{12006}, \alpha^{12098}, \alpha^{12265}, \alpha^{12301}, \alpha^{12525}, \alpha^{12732}, \alpha^{12753}, \alpha^{12905}, \alpha^{12923}, \alpha^{13035}, \alpha^{13149}, \alpha^{13225}, \alpha^{13234}, \alpha^{13290}, \alpha^{13347}, \alpha^{13385}, \alpha^{13446}, \alpha^{13465}, \alpha^{13505}, \alpha^{13525}, \alpha^{13535}, \alpha^{13540}, \alpha^{13608}, \alpha^{13671}, \alpha^{13797}, \alpha^{14049}, \alpha^{14074}, \alpha^{14090}, \alpha^{14553}, \alpha^{14602}, \alpha^{14603}, \alpha^{14635}, \alpha^{14750}, \alpha^{15028}, \alpha^{15412}, \alpha^{15554}, \alpha^{15561}, \alpha^{15659}, \alpha^{15661}, \alpha^{15725}, \alpha^{15844}, \alpha^{15955}\} \quad (16)$$

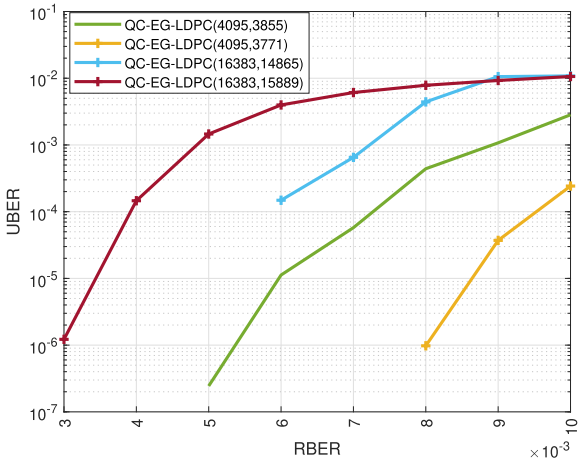


FIGURE 5. Performnce analysis of QC-EG-LDPC codes at higher RBER values.

Algorithm 7 Parity-Check Matrix Generation Using Finite Field Arithmetic

Input: Prime field size q, primitive element α, dimensions of Base matrix k, l
Output: Parity-check matrix H_{qc}
Step 1: Generate $GF(q)$ elements Table
Step 2: Select Subset Elements
$S_1 \leftarrow$ k elements from GF field
$S_2 \leftarrow$ l elements from GF field
Step 3: Construct Base Matrix
$\eta = 1$
$B_s \leftarrow \mathbf{0}_{k\times l}$ (initialize matrix)
for $a = 1$ to k **do**
  for $b = 1$ to l **do**
    $B_s(a,b) =$ mod $(S_1(a) + S_2(b), q)$
  end for
end for
Step 4: Generate Parity-Check Matrix
Initialize H_{qc} as an empty cell array
for row = 1 to k **do**
  for col = 1 to l **do**
    v=$(q-1)$-tuple of Base matrix element $B_s(row, col)$
    for $k = 1$ to $q-1$ **do**
      submatrix$(k,:) \leftarrow$ circshift$(v, k-1)$
    end for
    H_{qc}(row, col) $\leftarrow$ submatrix
  end for
end for
Return H_{qc}

results in a base matrix B_s of size 4×64. The generated base matrix has one zero element in each row. The 66−fold CPM dispersion of $B_s(4, 64)$ leads to H matrix of size 264×4224 with full rank of 264. To obtain the generator matrix from H, a square matrix D has to be chosen from H such that the rank of the matrix D is 264. Since H did not possess the D matrix, we applied masking on the H matrix. The base matrix B_s is multiplied by the masking matrix Z_{mask}. Here, the Hadamard matrix product is employed. The masking matrix is designed from $Z(4,\ 4k)$, where $k = 2l$, l is non negative integer [19]. The $Z(4, 8)$ is presented in 17. From this we have derived masking matrix $Z(4, 4 * 16) = Z(4, 64)$ by repeating $Z(4, 8)$ eight times. The 66-fold CPM dispersion of Hadamard matrix product $B_s(4, 64) \otimes Z(4, 64)$ resulted $H_{mask}(264, 4224)$ matrix with full rank. The columns from 133 to 396 from H_{mask} form the D_{mask} matrix with full rank of 264. Hence, D_{mask} is used to generate the generator matrix in systematic format (Algorithm 3).

$$Z(4,8) = \begin{bmatrix} 1&0&1&0&1&1&1&1 \\ 0&1&0&1&1&1&1&1 \\ 1&1&1&1&0&1&0&1 \\ 1&1&1&1&1&0&1&0 \end{bmatrix} \tag{17}$$

The parity check matrix H_{mask} has row weight of 47 and column weight of 3. The H_{mask} is a full rank matrix, the null space over $GF(2)$ results in (4224, 3960) QC-LDPC code with rate 0.93. The generator matrix for this code is generated using Algorithm 3. The density of the parity check matrix is 0.0111.

We have designed another QC-LDPC code with a rate of 0.968 in $GF(67)$. Here base matrix is of size 2×64 with $S_1 = 1, \alpha$ and $S_2 = 1, \alpha, \alpha^2,\ \alpha^3, \cdots\ \alpha^{63}$. The base matrix has one zero element in each row. The 66-fold CPM dispersion results in a full rank H matrix with row weights 63 and two column weights 1 and 2. The null space over GF (2) results in a QC-LDPC code with $n = 4224,\ k = 4092$. Here, the Generator matrix obtained by the systematic form (Algorithm 3). The density of the parity check matrix is 0.0149.

Another architecture we have considered is an SLC NAND developed by Micron Technology, having 32GB storage capacity [34]. The flash memory is organized into four planes, with a single plane comprising 2,048 blocks. A block consists of 128 pages, and every page includes 4,096 bytes for data, along with 224 bytes for parity check information storage. This results in a total of 4,320 bytes per page, combining both user data and overhead information. To facilitate ECC design, each page is divided into eight sectors. Hence, each sector contains 4320 bits of space.

To meet these architectural constraints, we have synthesized QC-LDPC codes from $GF(73)$. Let primitive element $\alpha = 5$ in $GF(73)$. All the elements of $GF(73)$ are generated using α. For the design purpose we choose the elements of subset $S_1 = \alpha,\ \alpha^2,\ \alpha^3,\ \alpha^4$ and elements of subset $S_2 = \alpha^5,\ \alpha^6, \cdots\ \alpha^{64}$. These subsets will result in a base matrix of size $(4, 60)$. The generated base matrix has a single zero element in each row. The 72-fold CPM dispersion of the base matrix results in a parity check matrix of size 288×4320, which is a full rank matrix. The null space of H results in (4320, 4032) QC-LDPC code. The row weight of the H matrix is 59, and the column weights of H are 3 and 4. The rate of the designed code is 0.93. Since the base matrix has zero elements in each row, the Zero matrix (ZM) in the parity check matrix results in row weight of 59 and two column weights of 3 and 4.

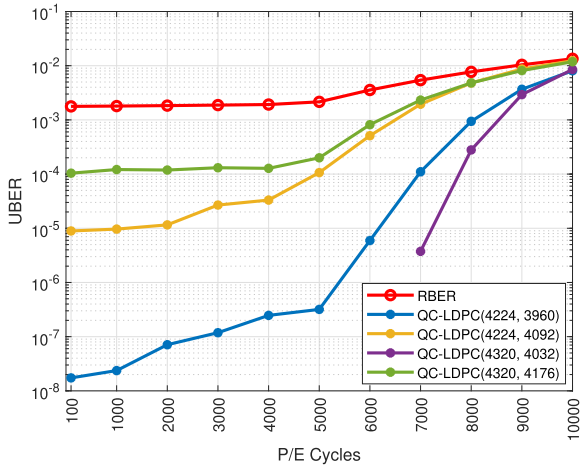


FIGURE 6. Performance analysis of QC-LDPC codes from *GF*(67) and *GF*(73) for different P/E cycles.

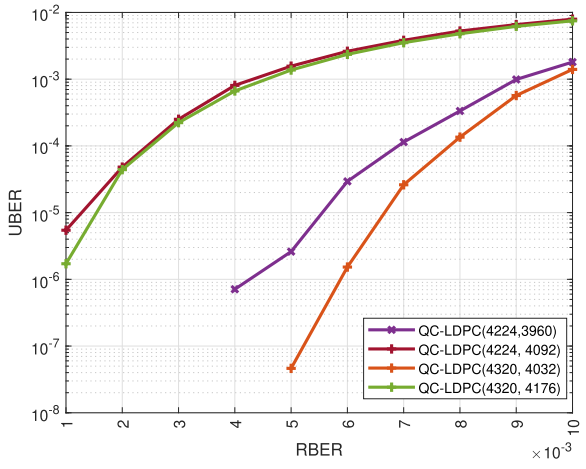


FIGURE 7. Performance analysis of QC-LDPC codes from *GF*(67) and *GF*(73) at higher RBER values.

We have also synthesized another code from $GF(73)$. Here, the elements of subset $S_1 = \alpha,\ \alpha^2$ and elements of subset $S_2 = \alpha^3,\ \alpha^4, \cdots \alpha^{62}$ will results in base matrix of size $(2, 60)$. Here, the base matrix has a single zero element in each row. The 72-fold CPM dispersion of the base matrix results in a full rank parity check matrix of size 144×4320. The null space over $GF(2)$ results in (4320, 4176) QC-LDPC code with code rate of 0.966. Since the base matrix has zero elements in each row, the Zero matrix (ZM) in the parity check matrix results in row weight of 59 and two column weights of 1 and 2. The density of H matrix is 0.0137.

The performance analysis of $(4224,\ 3960), (4224,\ 4092), (4320, 4032), (4320,\ 4176)$ QC-LDPC codes with respect different P/E cycles are plotted in Fig. 6. We have employed the MSA algorithm with five iterations for Monte-carlo simulations. The UBER plot for higher RBER values is also depicted in Fig. 7.

In summary, parameters of the synthesized LDPC codes in this paper are tabulated in Table 1. In Table 1, the performances of the designed codes at RBER of 1.8×10^{-3} are evaluated. It is the typical RBER value at 1000 P/E cycles in MLC NAND flash memory devices.

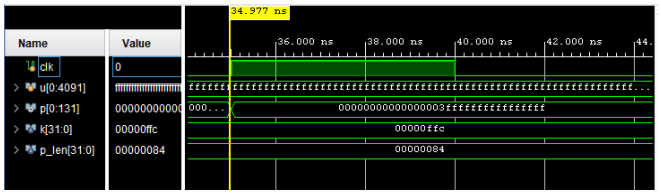


FIGURE 8. Simulation results for QC-LDPC (4224, 4092) encoder.

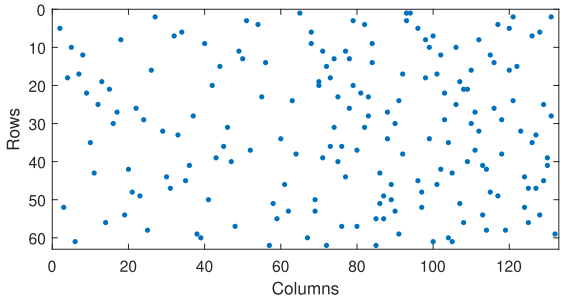


FIGURE 9. Parity matrix of QC-LDPC (4224, 4092) code.

Resource	Utilization	Available	Utilization %
LUT	2945	63400	4.65
FF	4356	126800	3.44
IO	134	210	63.81
BUFG	1	32	3.13

FIGURE 10. Implementation results of QC-LDPC (4224, 4092) encoder (Hard coded input).

We have also implemented these results using the Vivado 2022.2 tool with the Intel(R) Xeon(R) E-2224G CPU at 3.50GHz and a RAM size of 80GB. The simulation results, which present the parity check bits obtained for QC-LDPC (4224, 4092) code with message u possessing 4092 ones, are depicted in Fig. 8.

The designed encoder for QC-LDPC (4224,4092) has two CSRAA blocks with $G_{0:(t_{qc}-c_{qc})b,1}$ and $G_{0:(t_{qc}-c_{qc})b,2}$, which are loaded to register B with $b = 66$ bits at a time. The parity block of matrix G without CPM dispersion is presented in 9.

The implementation results for QC-LDPC (4224, 4092) in Xilinx Vivado 2022.2 are depicted in Fig. 10. Here we have hard coded input message inside the Verilog code, and parity bits are connected to output ports to meet the IO port requirement of the Artix-7 xc7a100tcsg324-1 device.

In the design of the MSA decoder for QC-LDPC codes, the full parity-check matrix H was not directly implemented in the Verilog code. Instead, an optimized representation was employed by storing only the indices of variable nodes connected to each check node. This reduced the effective size of the H matrix to a structure of dimensions equal to the number of check nodes by the maximum row weight. As a result, the storage requirements for the H matrix

TABLE 1. Parameters of synthesized LDPC codes.

Type	EG-LDPC	QC-EG-LDPC				QC-LDPC (finite field)			
Field	$EG^*(2,2^8)$	$EG^*(2,2^6)$	$EG^*(2,2^6)$	$EG^*(2,2^7)$	$EG^*(2,2^7)$	$GF(67)$	$GF(67)$	$GF(73)$	$GF(73)$
n	65535	4095	4095	16383	16383	4224	4224	4320	4320
k	58975	3855	3771	15889	14865	4092	3960	4176	4032
Code rate	0.9	0.94	0.92	0.97	0.907	0.95	0.94	0.9666	0.933
Density of H	0.0039	0.0156	0.0156	0.0078	0.0078	0.0149	0.0111	0.0137	0.0137
Row weight	256	64	64	128	128	63	47	59	59
Column Weight	256	3 and 4	5 and 6	3 and 4	19 and 20	1 and 2	3	1 and 2	3 and 4
Decoding Algorithm	BF (ite=10)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)	MSA (iter=5)
UBER for RBER 0.0018	0.0015	0	0	0	0	2.6×10^{-5}	0	3.2×10^{-5}	0

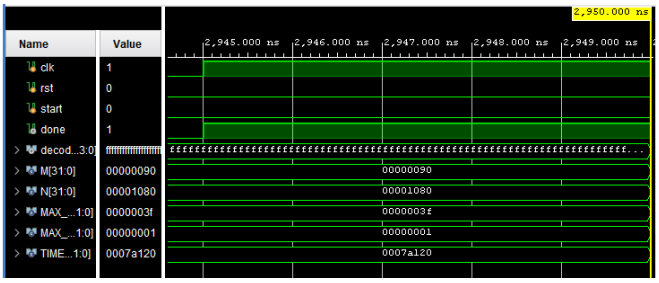


FIGURE 11. Simulation results of QC-LDPC (4224, 4092) decoder (Hard coded input).

Graph | **Table**

Resource	Estimation	Available	Utilization %
LUT	22935	63400	36.18
FF	73593	126800	58.04
IO	4228	210	2013.33
BUFG	1	32	3.13

FIGURE 12. Synthesis results of QC-LDPC (4224, 4092) decoder.

were significantly minimized, enabling a more efficient hardware implementation. The simulation results of decoding QC-LDPC (4224, 4092) are depicted in Fig 11. Here, the Verilog code for the MSA algorithm is simulated in Xilinx Vivado 2022.2 software.

The synthesis report for the QC-LDPC (4224, 4092) decoder is depicted in Fig 12.

Since the output message is of length 4224 bits, it exceeds the IO ports available in Artix-7 xc7a100tcsg324-1 device. Moving forward, resources utilized in the synthesis process carried out for the decoders of the synthesized codes in this paper are tabulated in Table 2. The 4224 decoded bits

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization...
LUT	28822	63400	45.46
FF	81177	126800	64.02
IO	202	210	96.19
BUFG	1	32	3.13

FIGURE 13. Implementation results for QC-LDPC (4224, 4092) decoder.

are stored internally in a register array inside the FPGA, then divided into 22 sequential chunks of 192 bits each, where a 5-bit chunk address input selects which chunk to present on the 192-bit output data bus at any given time. The implementation results obtained using the above method are presented in Fig. 13.

The QC-EG-LDPC codes employed in NAND flash memory devices will improve the device lifetime up to 10,000 P/E cycles as depicted in Fig. 4. The designed QC-EG-LDPC codes in this work give superior performance compared to other codes. However, the design procedure of QC-EG-LDPC codes restricts us to design codes in terms of prime two or powers of two. We can design the shortened code as per the architectural requirement; however, the decoding of the shortened code will require the full-length parity check matrix, which in turn increases the number of resources utilized. The design of QC-LDPC codes from a finite field allows us to synthesize codes satisfying various architectural environments. Choosing the appropriate Galois Field of any prime number that satisfies the architectural requirement is essential here.

TABLE 2. Resource utilization report from the synthesis process carried out for LDPC decoders using Xilinx Vivado 2022.2.

Type	QC-EG-LDPC		QC-LDPC (finite field)			
Field	$EG^*(2,2^6)$	$EG^*(2,2^6)$	$GF(67)$	$GF(67)$	$GF(73)$	$GF(73)$
(n, k)	(4095, 3855)	(4095, 3771)	(4224, 4092)	(4224, 3960)	(4320, 4176)	(4320, 4032)
LUT	64381	86158	22935	35002	76464	97115
FF	166399	231037	73593	107635	105349	173346
IO	4099	4099	4228	4228	4324	4324
BUFG	1	1	1	1	1	1

TABLE 3. Upper bound on minimum distance of the designed codes.

LDPC Codes		Upper bound on d_{min}
QC- EG	(4095, 3855)	21
	(4095, 3771)	49
QC codes from finite fields	(4224, 4092)	3
	(4224, 3960)	9
	(4320, 4176)	3
	(4320, 4032)	34

D. DETERMINATION OF MINIMUM DISTANCE IN QC-LDPC CODES

The accurate minimum distance of the designed LDPC code C can be determined by estimating the number of columns in the generator matrix G that sum to zero. This approach, however, is feasible only for codes with relatively small values of n and k, i.e., lower-dimensional generator matrices. In our work, all the designed codes have lengths greater than 4095. Therefore, determining the exact minimum distance was not feasible due to the high dimensionality of the generator matrix G. To estimate the upper bound on minimum distance, we have employed the approach outlined in [35].

Let C be a QC-LDPC code defined using polymomial parity check matrix $H(x)$.Then the upper bound on minimum hamming distance (d_{min}) is given by [35],

$$d_{\min}(\mathcal{C}) \leqslant \min_{\substack{\mathcal{A}\subseteq\delta \\ |\mathcal{A}|=\gamma+1}} \sum_{a\in\mathcal{A}} \mathrm{wt}\left(\mathrm{perm}\left(\boldsymbol{H}_{\mathcal{A}\setminus\delta}(x)\right)\right). \tag{18}$$

In (18) γ and δ are the number of row blocks and column blocks in H or the number of rows and columns in $H(x)$, respectively. A is subset of size $\gamma+1$ formed from all the combination of δ elemnts. Hence there $\binom{\delta}{\gamma+1}$ possible combinations. We have determined d_{min} for synthesized QC-LDPC Codes, and they are tabulated in Table 3. The Upper bound on the minimum distance of the designed QC-EG-LDPC (4095, 3771) code is a higher value compared to all the other codes. Hence, this code resulted in improved BER performance compared to all the other codes as depicted in Figs 4, 5, and 14.

The number of subsets formed for QC-EG-LDPC derived from $EG^*(2,2^7)$ are greater than 10^{10}. Hence, the d_{min} computation is not feasible.

1) COMPARISON WITH STATE-OF-THE-ART LDPC CODES

Our work has been motivated by the desire to synthesize and evaluate LDPC codes that can meet the architectural requirement constraints of flash memory devices while providing a code rate that is as high as possible. The performance of the codes synthesized in this paper has been evaluated by employing the MLC NAND flash memory channel described in Section II. We will provide a comparison of our codes with a few state-of-the-art LDPC codes that have been designed for similar applications below. In [15], authors have described the construction and implementation of QC-EG-LDPC (18396, 16416) code with a code rate of 0.892. This code can provide a UBER of 10^{-5} at RBER of 1.456×10^{-2}% with an average of 7.342 iterations for the decoding process. The QC-EG-LDPC (4095, 3771) code proposed in this paper, designed with NAND flash architecture in mind, provides a code rate of 0.92 and can deliver a UBER of 2×10^{-4} at RBER of 0.01 with five decoding iterations. In [38], the authors have designed a Regular QC-LDPC code with a codeword length of 18432 bits and an information length of 16384 bits. This code possesses a code rate of $R = 8/9 = 0.88$. The authors have reported that this code has a threshold RBER of 6.5×10^{-3}. The paper describes the threshold BER as the upper limit of error correction capability using LDPC codes with Hard Sensing Decoding. In our research, we have synthesized and characterized a QC-EG-LDPC (4095, 3771) code that possesses a threshold RBER of 8×10^{-3}

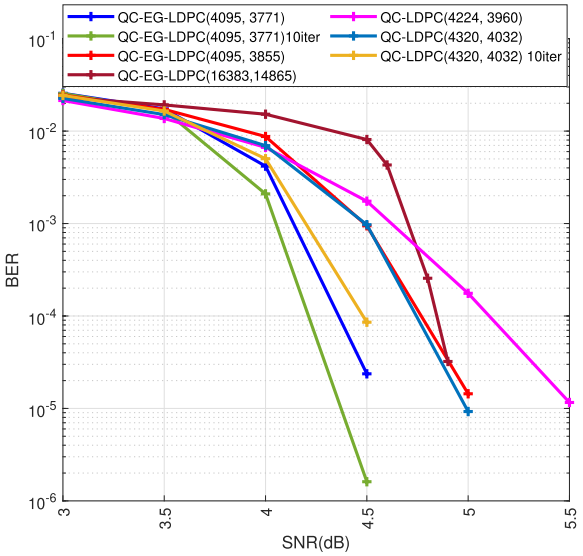


FIGURE 14. Evalution of LDPC codes with respect SNR(dB) in AWGN channel.

TABLE 4. Comparison to various LDPC codes constructed for NAND flash memory in literature.

		Proposed in this paper		**Proposed in this paper**		**[21]**	**[21]**	**[20]**	**[17]**	**[36]**
Code		QC-EG-LDPC		QC-LDPC		QC-LDPC	QC-LDPC	QC-LDPC	DA-LDPC	QC-LDPC
n		4095		4320		9280	34560	9280	18300	9216
k		3771		4032		8320	32832	8315	16470	8192
Code rate		0.92		0.933		0.9	0.95	0.896	0.9	0.88
Decoding Algorithm		MSA		MSA		layered MSA	layered MSA	TDMP	MSA	MSA
Decoding Iterations		5	10	5	10	8	8	8	20	4
UBER for RBER	10^{-2}	2.4×10^{-4}	4.2×10^{-5}	1.4×10^{-3}	5.3×10^{-4}	5×10^{-3}	2×10^{-2}	4×10^{-3}	10^{-4}	10^{-5}
	9×10^{-3}	3.7×10^{-5}	3.9×10^{-6}	5.6×10^{-4}	1.1×10^{-4}	2×10^{-3}	5.3×10^{-4}	2×10^{-3}	10^{-7}	5×10^{-6}

TABLE 5. Comparison of various LDPC codes constructed for NAND flash memory considering AWGN channel (SNR).

		Proposed in this paper		**Proposed in this paper**		**[22]**	**[23]**	**[37]**	**[37]**	**[16]**	**[16]**
Code		QC-EG-LDPC		QC-LDPC		QC-LDPC	LDPC	QC-LDPC	QC-LDPC	SC-GC-LDPC	GC-LDPC
n		4095		4320		8610	18900	2124	2016	37536	36848
k		3771		4032		8190	17010	1888	1792	33672	33320
Code rate		0.92		0.933		0.95	0.9	0.88	0.88	0.89	0.904
Decoding Algorithm		MSA		MSA		MSA	NMS	MSA	MSA	SPA/ NMS	SPA/ NMS
Decoding Iterations		5	10	5	10	12	20	50	50	20	20
BER for SNR (db)	4	4.1×10^{-3}	2×10^{-3}	6×10^{-3}	5×10^{-3}	-	10^{-2}	5×10^{-4}	6×10^{-4}	2×10^{-4}	10^{-3}
	4.5	2.3×10^{-5}	1.6×10^{-6}	9.7×10^{-4}	8.5×10^{-5}	10^{-2}	2×10^{-5}	2×10^{-6}	10^{-5}	-	-
	5	0	0	9.2×10^{-6}	0	2×10^{-4}	-	-	-	-	-

with a maximum of five decoding iterations. This allows us to reduce computational effort and decoding complexity while obtaining similar performance. Similarly, comparison with various other LDPC codes constructed for NAND flash memory application with respect to RBER is presented in Table 4. One of the strongest advantages of our work is that the proposed QC-LDPC codes are designed specifically to satisfy real NAND flash architectural constraints, including sector/page organization and parity overhead limitations. The QC LDPC code designs presented in [12] are of multi-rate type and utilize variable parity check spaces. Most NAND flash memory devices used in practise have fixed parity check space. The designed codes achieve very high code rates (exceeding 0.92), (a point not satisfied by previously proposed codes), and low decoding complexity with only five iterations. In [39], authors have focused on variable/check degree optimization for varying read precision and obtaining strong theoretical threshold results. However, they have not paid any attention to implementation (hardware) considerations. The work presented in [40] focuses on joint code–voltage optimization but does not provide hardware implementation results.

In Table 4, we have compared the performance of the designed code at higher RBER values, where the NAND flash device will not be in good use condition. We have chosen QC-EG-LDPC(4095, 3771) and QC-LDPC(4320, 4032) codes among the other proposed codes for the comparison. Proposed codes have a higher code rate except the LDPC code presented in [21], which results in lower UBER values. At decoding iterations of 10, the proposed codes result in improved performance in terms of UBER values.

In literature, the performance of LDPC codes designed for NAND flash devices has also been analyzed using the SNR (dB) parameter employing the Additive White Gaussian Noise(AWGN) channel model [16], [23]. In order to compare the performance of our codes with these codes, we have analyzed the performance of LDPC codes designed in our work over the AWGN channel. The performance of LDPC codes with respect to SNR(dB) in the AWGN channel is depicted in Fig. 14.

The $(n = 4095, k = 3771)$ QC-LDPC code with rate 0.92 synthesized by us in our paper has 0.25dB gain over the QC-LDPC codes designed in [22]. At SNR of 4.5dB the proposed QC-EG-LDPC(4095, 3771) has the BER of 2.3×10^{-5} with decoding iteration of 5 where as the the designed code QC-LDPC(4260, 4047) has BER of 3×10^{-3} and QC-LDPC(16999,16235) has BER of 10^{-3} with decoding iterations of 50 in [22]. Similarly, comparison to the

performance of other LDPC codes designed for NAND flash with respect to SNR (AWGN channel) is presented in Table 5. From the table, we can infer that the codes proposed in this paper result in better performance for the AWGN channel with a higher code rate and fewer decoding iterations. This makes them more suitable for NAND flash memory applications.

VI. CONCLUSION

Reliability remains a critical concern in NAND flash memory, particularly with respect to program/erase (P/E) cycles and overall device lifetime. To address this, powerful Error Correction Codes (ECC) are essential for improving the Raw Bit Error Rate (RBER) and extending device endurance. In this work, we have synthesized suitable ECC schemes, including EG-LDPC, QC-EG-LDPC, and QC-LDPC codes over finite fields. These codes are carefully tailored to align with the architectural constraints and high code rate requirements of NAND flash memory systems. All proposed codes maintain a code rate exceeding 0.9, with several designs achieving rates above 0.95, making them particularly effective in scenarios with minimal redundancy overhead. The designed QC-EG-LDPC (4095, 3771) code and QC-LDPC (4320, 4032) code when employed in NAND flash device will improve the life time by 10,000 P/E cycles as depicted in Fig. 4 and Fig. 6. We have also synthesized encoders and MSA decoders of the designed codes using Xilinx Vivado 2022.2 software. In our work, the parity check matrix density of QC-EG-LDPC codes is higher than QC-LDPC codes, resulting in decoders with a higher number of computational resources. By employing these advanced ECCs, we demonstrate a significant enhancement in both BER performance and P/E cycle endurance, thereby contributing to the long-term reliability of NAND flash storage.

REFERENCES

[1] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proc. IEEE*, vol. 105, no. 9, pp. 1666–1704, Sep. 2017.

[2] C.-W. Yoon, "The fundamentals of NAND flash memory: Technology for tomorrow's fourth industrial revolution," *IEEE Solid StateCircuits Mag.*, vol. 14, no. 2, pp. 56–65, Jun. 2022.

[3] K. Rajesh Shetty, U. Sripati, H. Prashantha Kumar, and B. Shankarananda, "Synthesis of BCH codes for enhancing data integrity in flash memories," in *Proc. 5th Int. Conf. Ind. Inf. Syst.*, Jul. 2010, pp. 119–124.

[4] G. Achala, U. S. Acharya, and P. Srihari, "On the design of SSRS and RS codes for enhancing the integrity of information storage in NAND flash memories," *IEEE Access*, vol. 11, pp. 73198–73217, 2023.

[5] K. R. Shetty, K. Ramakrishna, H. P. Kumar, and U. Sripati, "Design and construction of BCH codes for enhancing data integrity in multi level flash memories," *Int. J. Inf. Commun. Technol.*, vol. 4, no. 1, p. 40, Mar. 2012, doi: 10.1504/ijict.2012.045747.

[6] L. Dolecek and Y. Cassuto, "Channel coding for nonvolatile memory technologies: Theoretical advances and practical considerations," *Proc. IEEE*, vol. 105, no. 9, pp. 1705–1724, Sep. 2017.

[7] G. Achala, P. Srihari, and U. S. Acharya, "On the synthesis of channel codes for NAND flash devices in space application," in *Proc. IEEE Int. Conf. Electron., Comput. Commun. Technol. (CONECCT)*, Jul. 2024, pp. 1–6.

[8] H. Hu, H. Liu, K. Xi, K. Zhang, J. Zhang, and J. Liu, "Low-latency BCH-CRC decoder for 3D CT NAND flash memory applications," in *Proc. Silicon Nanoelectronics Workshop (SNW)*, Jun. 2021, pp. 1–2.

[9] G. Achala et al., "FPGA implementation of SSRS codes for NAND flash memory device," *IEEE Access*, vol. 12, pp. 140128–140143, 2024.

[10] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.

[11] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, Mar. 1999.

[12] L. Yin, X. Zhai, Y. Fang, and G. Han, "A multi-high-rate structured algebraic QC-LDPC code for 3D TLC NAND flash memory," *IEEE Trans. Commun.*, vol. 73, no. 11, pp. 10028–10043, Nov. 2025.

[13] M. Rowshan, M. Qiu, Y. Xie, X. Gu, and J. Yuan, "Channel coding toward 6G: Technical overview and outlook," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 2585–2685, 2024.

[14] M. P. C. Fossorier and S. Lin, "Soft-decision decoding of linear block codes based on ordered statistics," *IEEE Trans. Inf. Theory*, vol. 41, no. 5, pp. 1379–1396, Sep. 1995.

[15] L.-W. Liu, M.-H. Yuan, Y.-C. Liao, and H.-C. Chang, "A 38.64-Gb/s large-CPM 2-KB LDPC decoder implementation for NAND flash memories," *IEEE Open J. Circuits Syst.*, vol. 3, pp. 180–191, 2022.

[16] J. Mo, X. Zhai, M. Yu, Y. Fang, and G. Han, "SC-GC-LDPC for NAND flash memory: Construction and decoding," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 11, pp. 4167–4180, Nov. 2025.

[17] W. Shao, J. Sha, and C. Zhang, "Dispersed array LDPC codes and decoder architecture for NAND flash memory," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 8, pp. 1014–1018, Aug. 2018.

[18] P. Chen, K. Cai, and S. Zheng, "Rate-adaptive protograph LDPC codes for multi-level-cell NAND flash memory," *IEEE Commun. Lett.*, vol. 22, no. 6, pp. 1112–1115, Jun. 2018.

[19] S. Lin and D. J. Costello, *Error Control Coding*. Pearson, 2021.

[20] M.-H. Chung, Y.-M. Lin, C.-Z. Zhan, and A. Y. Wu, "Cost-effective scalable QC-LDPC decoder designs for non-volatile memory systems," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2013, pp. 2625–2628.

[21] Y. Lin, H.-T. Li, M.-H. Chung, and A.-Y. Wu, "Byte-reconfigurable LDPC codec design with application to high-performance ECC of NAND flash memory systems," in *Proc. IEEE Trans. Circuits Syst. I, Reg. Papers*, Jun. 2015, vol. 62, no. 7, pp. 1794–1804.

[22] W. Zhang, J. Kang, Z. Dong, and Y. Zhu, "RS-LDPC concatenated coding for NAND flash memory: Designs and reduction of short cycles," in *Proc. IEEE 3rd Int. Conf. Inf. Commun. Signal Process. (ICICSP)*, Sep. 2020, pp. 342–347.

[23] K.-C. Ho, C.-L. Chen, and H.-C. Chang, "A 520k (18900, 17010) array dispersion LDPC decoder architectures for NAND flash memory," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 4, pp. 1293–1304, Apr. 2016.

[24] C. A. Aslam, Y. L. Guan, and K. Cai, "Decision-directed retention-failure recovery with channel update for MLC NAND flash memory," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 353–365, Jan. 2018.

[25] G. Dong, N. Xie, and T. Zhang, "On the use of soft-decision error-correction codes in NAND flash memory," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 58, no. 2, pp. 429–439, Feb. 2011.

[26] Z. Mei, K. Cai, and X. He, "Deep learning-aided dynamic read thresholds design for multi-level-cell flash memories," *IEEE Trans. Commun.*, vol. 68, no. 5, pp. 2850–2862, May 2020.

[27] W. E. Ryan, S. Lin, and S. G. Wilson, *Channel Codes: Classical and Modern*. Cambridge, U.K.: Cambridge Univ. Press, 2024.

[28] L. Rudolph, "A class of majority logic decodable codes (corresp.)," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 2, pp. 305–307, Apr. 1967.

[29] I. Reed, "A class of multiple-error-correcting codes and the decoding scheme," *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 38–49, Sep. 1954.

[30] L. Zhang, S. Lin, K. Abdel-Ghaffar, Z. Ding, and B. Zhou, "Quasi-cyclic LDPC codes on cyclic subgroups of finite fields," *IEEE Trans. Commun.*, vol. 59, no. 9, pp. 2330–2336, Sep. 2011.

[31] D. Kim, K. R. Narayanan, and J. Ha, "Symmetric block-wise concatenated BCH codes for NAND flash memories," *IEEE Trans. Commun.*, vol. 66, no. 10, pp. 4365–4380, Oct. 2018.

[32] J.-Y. Kim, S.-H. Park, H. Seo, K.-W. Song, S. Yoon, and E.-Y. Chung, "NAND flash memory with multiple page sizes for high-performance storage devices," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 2, pp. 764–768, Feb. 2016.

[33] Micron Technology. (2005). *TN-29-07: Small-Block Vs. Large-Block NAND Flash Devices Array Organization*. [Online]. Available: https://www.micron.com/support/~/media/74C3F8B1250D4935898DB7FE79EB56E7.ashx

[34] Micron Technology. (2009). *16Gb, 32Gb, 64Gb, 128Gb Asynchronous/Synchronous NAND Features*. [Online]. Available: https://datasheetspdf.com/pdf/697298/Micron/MT29F32G08AFABA/1

[35] R. Smarandache and P. O. Vontobel, "Quasi-cyclic LDPC codes: Influence of proto- and tanner-graph structure on minimum Hamming distance upper bounds," *IEEE Trans. Inf. Theory*, vol. 58, no. 2, pp. 585–607, Feb. 2012.
[36] C. Zhang, J. Mo, Z. Lian, and W. He, "High energy-efficient LDPC decoder with AVFS system for NAND flash memory," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–4.
[37] H. Li, X. Jiang, Z. Yu, and W. Zheng, "A novel LDPC construction scheme for NAND flash memory," in *Proc. 5th Int. Conf. Circuits, Syst. Simul. (ICCSS)*, May 2022, pp. 21–26.
[38] M. Zhang et al., "ALCod: Adaptive LDPC coding for 3-D NAND flash memory using inter-layer RBER variation," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 1068–1081, Nov. 2023.
[39] K. Vakilinia, D. Divsalar, and R. Wesel, "Optimized degree distributions for binary and non-binary LDPC codes in flash memory," in *Proc. Int. Symp. Inf. Theory Appl.*, Oct. 2014, pp. 6–10.
[40] C. Duangthong, W. Phakphisut, and P. Wardkein, "Efficient design of read voltages and LDPC codes in NAND flash memory using density evolution," *IEEE Access*, vol. 11, pp. 74420–74437, 2023.

G. ACHALA (Graduate Student Member, IEEE) received the B.E. degree in electronics and communication engineering and the M.Tech. degree in digital communication from Visvesvaraya Technological University (VTU), Belgaum, Karnataka, in 2012 and 2016, respectively. She is currently pursuing the Ph.D. degree in electronics and communication engineering with the National Institute of Technology Karnataka. She also did her internship at Tejas Networks, Bengaluru. Her research interests include error control coding, radar signal processing, and wireless communication.

U. SHRIPATHI ACHARYA (Senior Member, IEEE) received the Ph.D. degree in error control codes and their application in wireless communication from Indian Institute of Science in 2005. He is currently a Professor with the Department of Electronics and Communication, National Institute of Technology Karnataka. He has led several funded research projects in the area of wireless communication, free space optical communication, and professional electronics. His research interests are the application of error control codes to communication and data storage, underwater free space communication, wireless and MIMO communication, and professions electronics. He is a fellow of the Institution of Engineers (India).

PATHIPATI SRIHARI (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering from Sri Venkateswara University, the master's degree in communications engineering and signal processing from the University of Plymouth, U.K., and the Ph.D. degree in radar signal processing from Andhra University in 2012. He is currently working as an Assistant Professor with the National Institute of Technology Karnataka, Surathkal, India. He worked as a Visiting Assistant Professor at McMaster University, Canada, in 2014. His research interests are radar target tracking, radar waveform design, and efficient DSP algorithms for radar applications. He is a Senior Member of ACM. He is a fellow of IETE and a member of IEICE, Japan. He received the 2010 IEEE Asia–Pacific Outstanding Branch Counselor Award. He received a Young Scientist Award from the Department of Science and Technology (DST), New Delhi, to carry out a sponsored research project titled "Development of efficient target tracking algorithms in the presence of ECM."

LINGA REDDY CENKERAMADDI (Senior Member, IEEE) received the master's degree in electrical engineering from Indian Institute of Technology Delhi (IIT Delhi), New Delhi, India, in 2004, and the Ph.D. degree in electrical engineering from Norwegian University of Science and Technology (NTNU), Trondheim, Norway, in 2011.

He worked at Texas Instruments on mixed-signal circuit design before joining the Ph.D. Program at NTNU. After finishing the Ph.D. degree, he worked on radiation imaging for an atmosphere–space interaction monitor (ASIM mission to the international space station) at the University of Bergen, Bergen, Norway, from 2010 to 2012. He is currently the Leader of the Autonomous and Cyber-Physical Systems (ACPS) Research Group and a Professor at the University of Agder, Grimstad, Norway. He has co-authored over 210 research publications that have been published in prestigious international journals and standard conferences in the research areas of Internet of Things (IoT), cyber-physical systems, autonomous systems, robotics and automation involving advanced sensor systems, computer vision, thermal imaging, LiDAR imaging, radar imaging, wireless sensor networks, smart electronic systems, advanced machine learning techniques, and connected autonomous systems, including drones/unmanned aerial vehicles (UAVs), unmanned ground vehicles (UGVs), unmanned underwater systems (UUSs), 5G- (and beyond) enabled autonomous vehicles, socio-technical systems like urban transportation systems, smart agriculture, and smart cities. He is also quite active in medical imaging. Several of his master's students won the best master thesis award in information and communication technology (ICT). He is also a member of ACM and the editorial boards of various international journals and the technical program committees of several IEEE conferences. He serves as a reviewer for several reputed international conferences and IEEE journals. He is the Principal Investigator and a Co-Principal Investigator of many research grants from Norwegian Research Council.